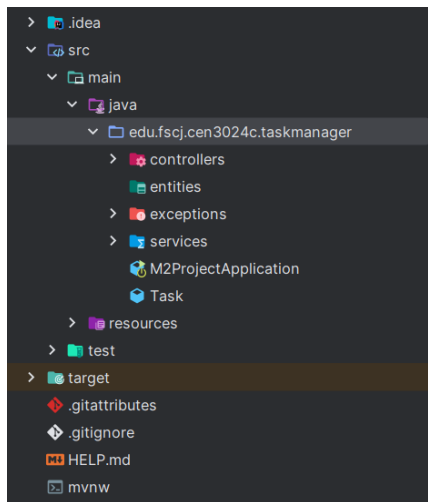


CEN3024C Module 3 Project

For this project we will continue working with our task management application from Module 2 by adding a database and using Spring JPA to persist our data. We will continue to use Spring Web using the classes developed in Module 2.

Now that your project is growing, notice that I have added packages to hold the classes you created in Module 2.



Note: In addition to specifying the Spring Boot dependencies show below, you will need to load the “JPA Buddy” plug-in in IntelliJ as follows: File > Settings > Plugins; search for "JPABuddy" then restart IntelliJ.

Using the Module 2 project solution as a starting point, add the following dependencies to the POM file and sync the project using the Maven sync option.

SQL/Spring Data JPA

SQL/PostgreSQL Driver

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

The PostgreSQL DB will be created using the command line psql utility, once it is created all other DB interactions and configurations are handled by IntelliJ and its Data Connection tool.

PostgreSQL tips:

- Names which use upper case must be in double quotes
- Verify psql is in your search path by running "where psql". If the utility is not found you will need to add the PostgreSQL bin folder (e.g., c:"Program Files"\PostgreSQL\16\bin") to your execution search path.
- Use lower case (default) for table names so JPA/Hibernate annotation works
- **psql -U postgres -d postgres** connects to the default database ("postgres")
- The psql password on Horizon is P@ssw0rd1
- \l to list databases
- \c dbname to connect to a db
- \dt to list tables and relationships (reports everything as "relations")
- \dt+ for extended details
- \dn to list schemas
- \q to quit

Create the database using psql:

```
D:\> c:"Program Files"\PostgreSQL\16\bin\psql -U postgres -d postgres
Password for user postgres
```

```
psql (16.1)
```

```
WARNING: Console code page (437) differs from Windows code page (1252)
        8-bit characters might not work correctly. See psql reference
        page "Notes for Windows users" for details.
```

```
Type "help" for help.
```

```
postgres=# \l
```

```
    List of databases
```

Name		Owner	...
postgres		postgres	...

```
...
```

```
(3 rows)
```

```
postgres=# CREATE DATABASE taskmanager_db;
CREATE DATABASE
```

```
postgres=# \l
```

```
    List of databases
```

Name		Owner	...
postgres		postgres	...
taskmanager_db		postgres	...

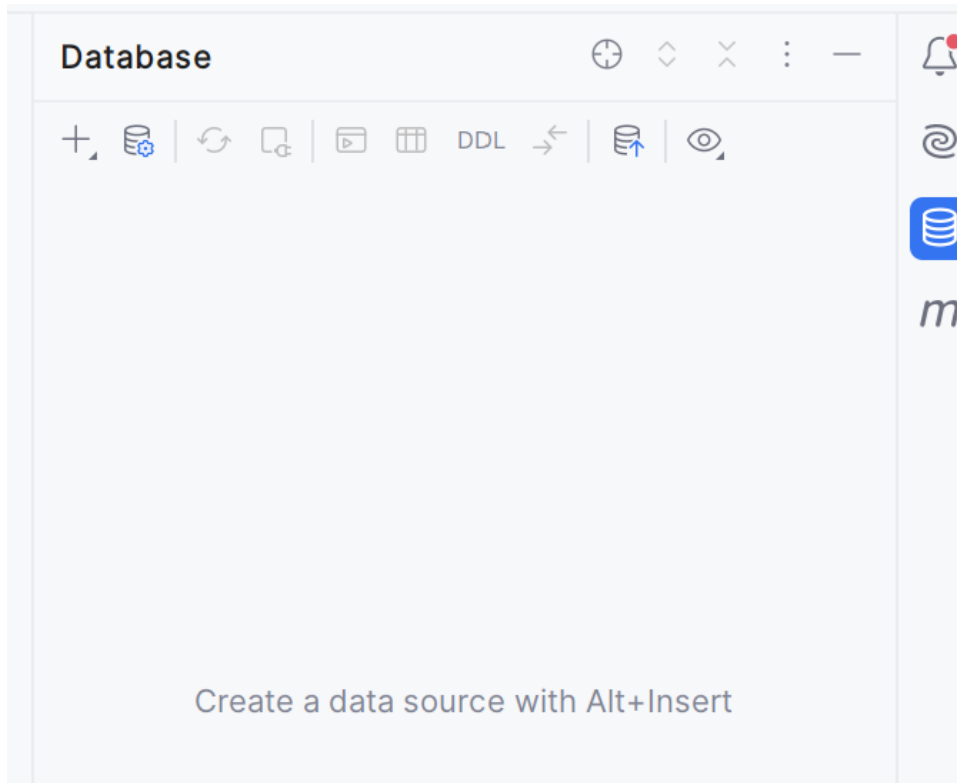
```
...
```

```
(4 rows)
```

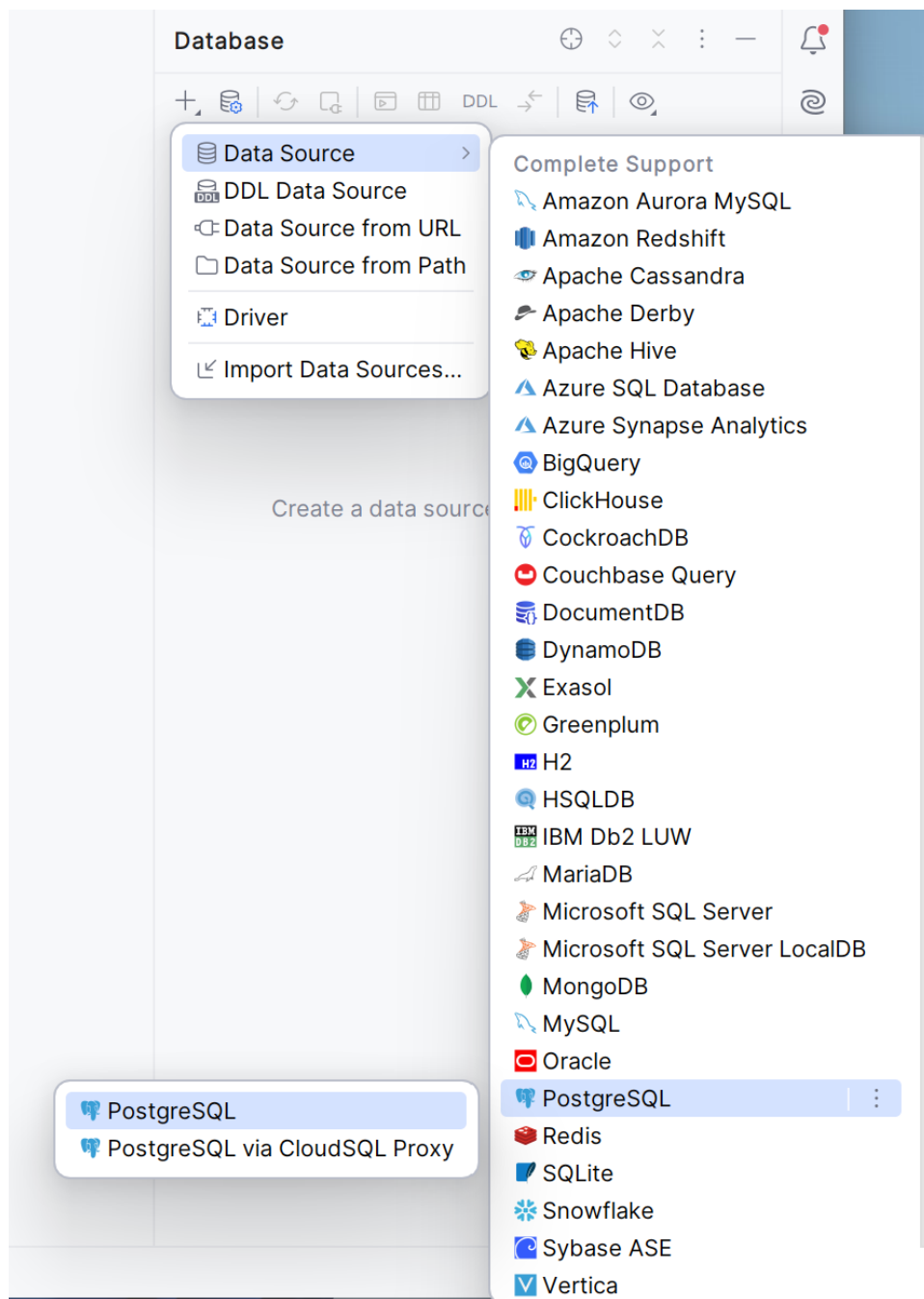
postgres=# \q

D:\fscj\CEN3024C-SoftwareDevI\cen3024c-Fall124\assignments>

- Open the Database tool in IntelliJ:



- Create a new data source using PostgreSQL:



(**note:** the Name: field refers to the name of the connection; specify the actual database name in the Database field)

Now we are going to turn our Task class into a JPA entity that will generate a table in our database

Notice the constructor has been removed and annotations have been added to the class

- @Entity – specifies the class as a JPA entity
- @GeneratedValue – specifies the auto generated Id value
- @Id – specifies the Id of the entity
- @Column – specifies that the attribute is a column
 - Notice the name, length, and nullable options on the column variables
 - The default length is 255
 - The default to nullable is true

```
package edu.fscj.cen3024c.taskmanager.entities;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
public class Task {
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    @Id
```

```
    @Column(name = "taskid", nullable = false)
```

```
    private Integer id;
```

```
    @Column(name = "title", nullable = false)
```

```
    private String title;
```

```
    @Column(name = "description")
```

```
    private String description;
```

```
    @Column(name = "status", length = 11, nullable = false)
```

```
    private String status; // PENDING, IN_PROGRESS, COMPLETED
```

```
    @Column(name = "due_date", length = 10)
```

```
    private String dueDate; // YYYY-MM-DD
```

```
    public Integer getId() {
```

```
        return id;
```

```
    }
```

```
    public void setId(Integer id) {
```

```
        this.id = id;
```

```

    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public String getStatus() {
        return status;
    }

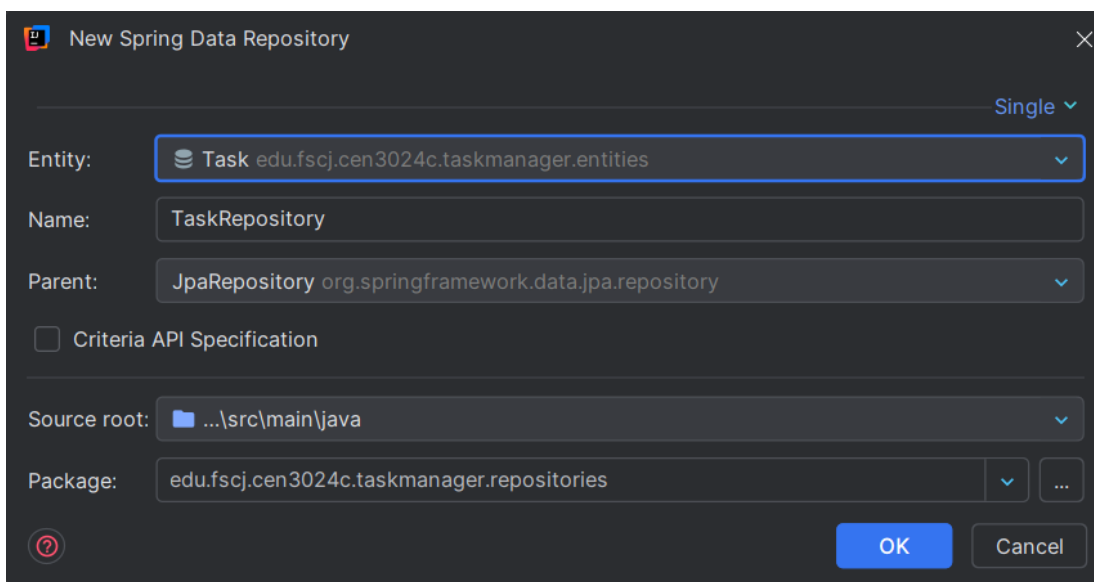
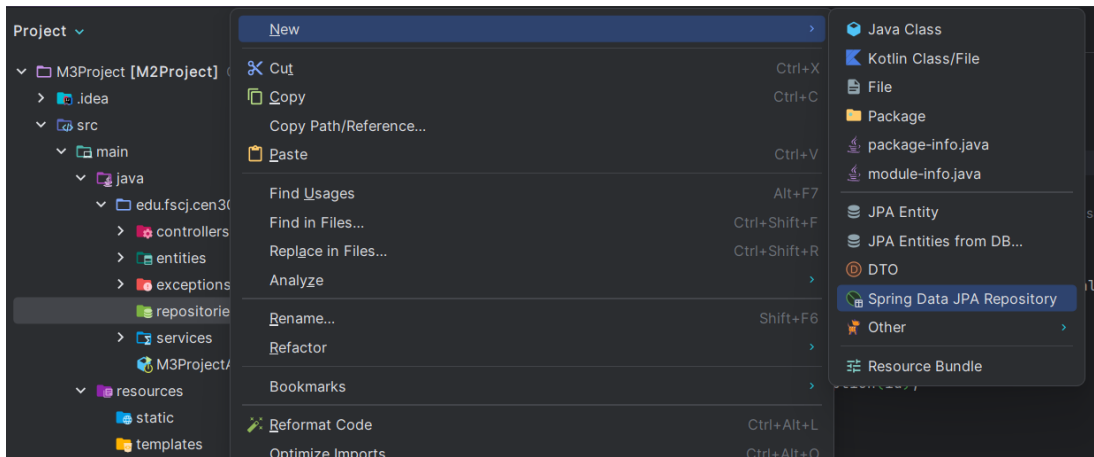
    public void setStatus(String status) {
        this.status = status;
    }

    public String getDueDate() {
        return dueDate;
    }

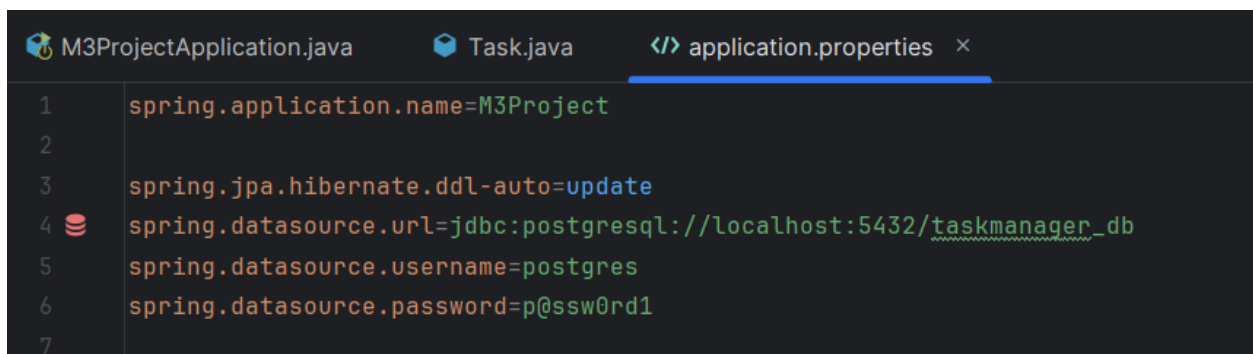
    public void setDueDate(String dueDate) {
        this.dueDate = dueDate;
    }
}

```

- Implement a TaskRepository **interface** that extends JpaRepository (an interface in Spring Data that provides CRUD operations for JPA entities without boilerplate code):

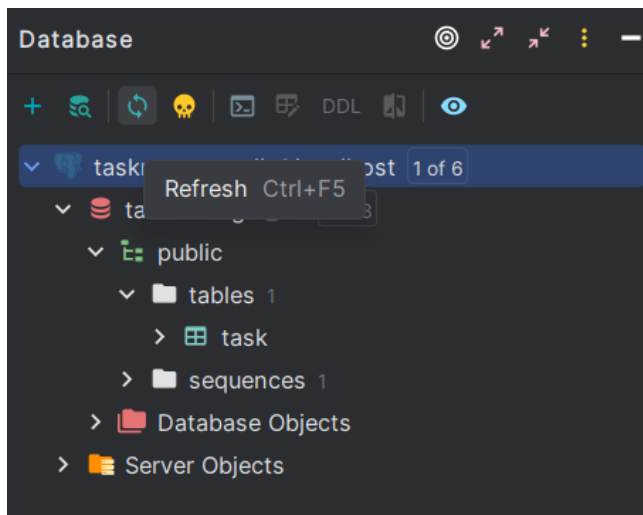


Once your task entity and repository is created add the following to your resources > application.properties file

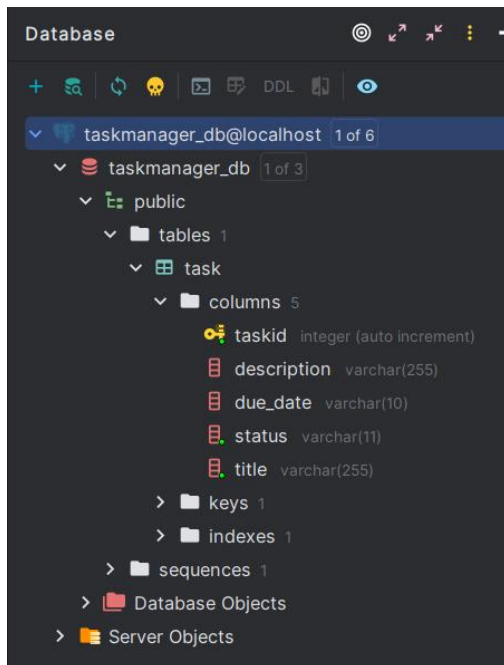


Run your program

Refresh your database manager on the right (Right-click and refresh)



Take a screenshot that verifies your table was generated properly from the Entity



Run the Curl commands to test and verify; take screenshots of your results.

Submit your project solution and screenshots to the repository.